Formal Modeling on Propagator Networks

The Landscape of Temporal Logic, Fixpoint Theory, and Formal Verification — and How Propagators Embed It All

The Prologos Project - from Zachary Larson

2026-03-14

Contents

1 Abstract **3**

2 1. The Modal μ-Calculus **4**
2.1 1.1 Foundations 4
2.2 1.2 Expressiveness: The Janin-Walukiewicz Theorem 5
2.3 1.3 The Alternation Hierarchy 5
2.4 1.4 Parity Games: The Decision Procedure 6
2.5 1.5 The Automata-Logic-Games Triangle 6
2.6 1.6 Extensions 7
2.6.1 Coalgebraic μ-Calculus 7
2.6.2 Quantitative and Vectorial μ-Calculus 7
2.6.3 Fixpoint Logic with Chop (FLC) 7

3 2. The Temporal Logic Landscape **7**
3.1 2.1 The Classical Core: LTL, CTL, CTL* 7
3.2 2.2 Signal Temporal Logic (STL) 8
3.3 2.3 Metric Temporal Logic (MTL) 9
3.4 2.4 HyperLTL and Hyperproperties 10
3.5 2.5 Probabilistic Temporal Logics 11
3.6 2.6 Spatial and Spatio-Temporal Logics 11
3.7 2.7 Quantitative and Weighted Temporal Logics 11

4 3. Fixpoint Theory and Propagator Networks **12**
4.1 3.1 The Knaster-Tarski Foundation 12
4.2 3.2 Approximation Fixpoint Theory (AFT) 12

4.3 3.3 Belnap's Four-Valued Logic and Bilattices 13
4.4 3.4 Mixed Least/Greatest Fixpoints 14
4.5 3.5 Domain Theory and Propagators 15
4.6 3.6 Coinduction and Greatest Fixpoints 15
4.7 3.7 Fixpoint Games on Continuous Lattices 16
4.8 3.8 Widening and Narrowing 17

5 4. Runtime Verification and Monitoring 17
5.1 4.1 Three-Valued Semantics (LTL3) 17
5.2 4.2 Monitoring Tools and Architectural Patterns 18
5.3 4.3 Distributed Runtime Verification 19
5.4 4.4 Predictive Runtime Verification 20
5.5 4.5 Shield Synthesis 20

6 5. Session Types, Linear Logic, and Verification 21
6.1 5.1 The Curry-Howard Correspondence for Sessions 21
6.2 5.2 Recursive Session Types: μ vs ν 21
6.3 5.3 Multiparty Session Types 22
6.4 5.4 Session Types and Model Checking 22
6.5 5.5 Actris: Session Types Meet Separation Logic 23

7 6. Formal Verification Frontiers 23
7.1 6.1 Separation Logic and Iris 23
7.2 6.2 Refinement Types 24
7.3 6.3 Algebraic Effects and Handlers 24
7.4 6.4 Reactive Synthesis 24
7.5 6.5 Coalgebraic Semantics 25
7.6 6.6 Capability-Based Security Verification 25

8 7. The Propagator Network as Universal Verification Substrate 26
8.1 7.1 The Fundamental Observation 26
8.2 7.2 What Prologos Already Has (Infrastructure Inventory) . . 27
8.3 7.3 What Would Complete the Picture 28
8.3.1 7.3.1 Greatest Fixpoint Support (Descending Cells) . . 28
8.3.2 7.3.2 Temporal Property Specification Language . . . 28
8.3.3 7.3.3 Approximation Fixpoint Theory Integration . . . 29
8.3.4 7.3.4 Runtime Monitoring Compilation 29
8.3.5 7.3.5 Quantitative Verification Support 29
8.4 7.4 The Verification Layer Cake (Extended) 30

9 8. Cross-Cutting Themes **30**
9.1 8.1 The Lattice Universality Principle 30
9.2 8.2 The Duality of Safety and Liveness 31
9.3 8.3 Static vs. Dynamic Verification 31
9.4 8.4 The Abstraction-Refinement Paradigm 32
9.5 8.5 Resource Discipline as Universal Theme 32

10 9. Research Program **33**
10.1 9.1 Immediate (Infrastructure Extensions) 33
10.2 9.2 Medium-Term (New Verification Capabilities) 33
10.3 9.3 Long-Term (Research Frontiers) 33

11 10. References **34**
11.1 Modal μ-Calculus and Fixpoint Theory 34
11.2 Temporal Logics 35
11.3 Hyperproperties and Information Flow 35
11.4 Probabilistic and Quantitative Verification 35
11.5 Runtime Verification 36
11.6 Approximation Fixpoint Theory and Bilattices 36
11.7 Abstract Interpretation 37
11.8 Session Types and Linear Logic 37
11.9 Separation Logic 37
11.10 Refinement Types and Verification 38
11.11 Algebraic Effects 38
11.12 Reactive Synthesis 38
11.13 Capability Security 38
11.14 Coalgebraic Methods 38
11.15 Domain Theory and Coinduction 39
11.16 Propagator Networks 39
11.17 Prologos Project Internal Documents 39

1 Abstract

This document surveys the landscape of formal modeling, temporal logic, and verification, with a sustained focus on how propagator networks serve as a universal computational substrate for these formalisms. The core observation, developed in our earlier research (`PROPAGATORS_AS_MODEL_CHECKERS.org`), is that model checking, abstract interpretation, and temporal reasoning are all *monotone fixpoint computations over complete lattices* — and propagator networks are precisely the engine for such computations.

We cast a wide net: from the modal μ-calculus and its alternation hierarchy through signal temporal logic and continuous-domain verification; from Approximation Fixpoint Theory and bilattices through coinduction and greatest fixpoints; from runtime verification and shield synthesis through reactive synthesis and separation logic. In each case, we identify the lattice-theoretic core and trace the connection to propagator network computation. The document concludes with a synthesis of how these formalisms map to Prologos's existing and planned infrastructure, and a research program for bringing formal verification from expert-only tools to type-system-level accessibility.

This is a stage-0 research document: it maps the territory and identifies implementation tracks, but does not commit to specific implementation timelines.

2 1. The Modal μ-Calculus

2.1 1.1 Foundations

The modal μ-calculus (Kozen 1983) is the most expressive logic in the temporal/modal family that retains decidable model checking. It extends propositional modal logic with explicit least (μ) and greatest (ν) fixpoint operators:

$\phi ::= \mathrm{p} \mid \neg\, \phi \mid \phi \wedge \psi \mid \phi \vee \psi$

$$\begin{array}{ll} \langle\ \mathrm{a}\ \rangle\ \phi & [\mathrm{a}]\ \phi \\ \mathrm{X} & \mu\ \mathrm{X}.\ \phi(\mathrm{X}) \\ \nu\ \mathrm{X}.\ \phi(\mathrm{X}) & \end{array}$$

where X occurs positively in $\phi(\mathrm{X})$ (under an even number of negations).

Semantics: Formulas are interpreted over labeled transition systems (S, $\{\mathrm{R_a}\}_{\mathrm{a}\ \in\ \mathrm{Act}}$, L). The denotation $\phi \subseteq Sisasubsetofstates.Thefixpointoperatorshaveset-theoreticsemanticsviatheKnaster-Tarskitheorem:$

- μ X. $\phi(\mathrm{X}) = \bigcap\backslash\ \mathrm{T} \subseteq S \mid F(T) \subseteq T\backslash$ (least pre-fixpoint = least fixpoint for monotone F)
- ν X. $\phi(\mathrm{X}) = \bigcup\backslash\ \mathrm{T} \subseteq S \mid T \subseteq F(T)\backslash$ (greatest post-fixpoint = greatest fixpoint)

where $\mathrm{F(T)} = \phi(\mathrm{X} := \mathrm{T})$ is monotone by the positivity requirement.

When the state space is finite, fixpoints are computable by Kleene iteration: μ X. $\phi(\mathrm{X}) = \bigcup_{\mathrm{i}\ \geq\ 0} F^{\mathrm{i}}(\emptyset)$, converging in at most |S| steps. Dually, ν X. $\phi(\mathrm{X}) = \bigcap_{\mathrm{i}\ \geq\ 0} F^{\mathrm{i}}(S)$.

Key references:

- Kozen, D. "Results on the propositional μ-calculus." *TCS* 27(3), 1983.
- Walukiewicz, I. "Completeness of Kozen's axiomatisation." *Information and Computation* 157(1-2), 2000.

2.2 1.2 Expressiveness: The Janin-Walukiewicz Theorem

The μ-calculus has exactly the right expressiveness for model checking:

Theorem (Janin-Walukiewicz 1996): The modal μ-calculus is expressively equivalent to the bisimulation-invariant fragment of monadic second-order logic (MSO) over trees.

This places the μ-calculus at a precise point in the logical landscape: it captures everything that MSO can say about transition systems, up to bisimulation equivalence. Since bisimulation is the natural equivalence for concurrent systems, the μ-calculus is the "right" logic for process verification.

2.3 1.3 The Alternation Hierarchy

The alternation depth of a μ-calculus formula measures the nesting depth of mutually recursive least and greatest fixpoint operators. Bradfield (1996) and Lenzi (1996) independently proved:

Theorem (Bradfield 1996): The modal μ-calculus alternation hierarchy is **strictly infinite**. For every n, there exist properties expressible at alternation depth n+1 but not at depth n.

The hierarchy levels correspond to increasing complexity of temporal properties:

Level	μ-Calculus Pattern	Temporal Meaning	Example
0	p (no fixpoint)	State property	"this state is labeled p"
1	μ X. ϕ or ν X. ϕ	Safety/reachability	AG p, EF p
2	ν X. μ Y. ϕ(X,Y)	Response/persistence	AG(p $\rightarrow$ AF q), AGEF p
3+	Deeper alternation	Fairness, complex liveness	Streett/Rabin conditions

For Prologos: Our current session type checker operates at Level 1 (safety via contradiction detection, reachability via completeness checking). Level 2 properties (response: "every request gets a reply") require nested fixpoints — computationally feasible in propagator networks but needing an interface for specification. Level 3+ properties appear rarely in session type contexts but are needed for fairness in multi-party protocols.

2.4 1.4 Parity Games: The Decision Procedure

Model checking μ-calculus formulas reduces to solving *parity games* — two-player infinite games where vertices are colored with natural numbers (priorities) and the winner is determined by the parity (even/odd) of the highest priority seen infinitely often.

Theorem (Emerson-Jutla-Sistla 1993): The μ-calculus model checking problem is equivalent to solving parity games.

The complexity of parity games was a major open problem for decades:

- The problem lies in NP $\cap$ coNP (Emerson-Jutla-Sistla 1993)
- It is in UP $\cap$ coUP (Jurdzinski 1998)
- **Quasipolynomial-time algorithms** were discovered by Calude, Jain, Khoussainov, Li, and Stephan (STOC 2017), resolving a long-standing conjecture. The algorithm runs in time $n^{O(\log d)}$ where n is the game size and d is the number of priorities.
- Jurdzinski and Lazic (LICS 2017) gave a succinct progress measure algorithm also achieving quasipolynomial time.

For propagators: Parity games correspond to mixed lfp/gfp computations with alternating fixpoints. The game-theoretic view suggests that propagator networks computing nested fixpoints can be understood as strategies in parity games, with the scheduler playing the role of the game solver.

2.5 1.5 The Automata-Logic-Games Triangle

A deep structural equivalence connects three formalisms:

```
  Alternating Parity Tree Automata
            /                    \
           /                      \
Modal -Calculus  ←----→   Parity Games
```

- μ-calculus formulas translate to alternating parity tree automata (Wilke 2001)
- Emptiness of alternating parity tree automata reduces to parity games
- Parity games can be encoded as μ-calculus model checking problems

This triangle means results in any one formalism transfer to the others. The quasipolynomial breakthrough for parity games immediately gives quasipolynomial μ-calculus model checking and automata emptiness.

2.6 1.6 Extensions

2.6.1 Coalgebraic μ-Calculus

Cirstea, Kupke, and Pattinson generalize the μ-calculus from Kripke frames to coalgebras for arbitrary endofunctors F : Set → Set. The key innovation is *predicate liftings* λ : Q^n → Q ˆ F that replace the modal operators ⟨ a ⟩ and [a] with functorial generalizations. This captures:

- Probabilistic modal logic (F = distribution functor)
- Graded modal logic (F = multiset/bag functor)
- Coalition logic (F = coalition functor)

The coalgebraic perspective is categorically dual to inductive definitions — where least fixpoints are initial algebras, greatest fixpoints are terminal coalgebras. This connects directly to the coinductive reasoning Prologos needs for recursive session types.

2.6.2 Quantitative and Vectorial μ-Calculus

Fischer and Grädel (LICS 2023) develop a *vectorial* μ-calculus where fixpoint variables range over tuples of sets, enabling systems of mutually recursive fixpoint equations. This corresponds to multi-cell propagator interactions where cells mutually constrain each other.

Lattice-valued μ-calculus replaces the Boolean powerset 2^S with an arbitrary complete lattice L, yielding L-valued satisfaction. This is precisely the generalization that propagator networks already implement: cells hold lattice values, not Boolean sets.

2.6.3 Fixpoint Logic with Chop (FLC)

Müller-Olm (1999) extends the μ-calculus with a sequential composition operator ("chop"), enabling specification of properties that depend on the *sequential structure* of computations. This is relevant for session types, which are inherently sequential protocols.

3 2. The Temporal Logic Landscape

3.1 2.1 The Classical Core: LTL, CTL, CTL*

Three fundamental temporal logics form the core of the landscape:

LTL (Pnueli 1977) reasons about individual computation paths with operators X (next), U (until), F (eventually), G (always). It captures exactly the star-free ω-regular languages (Kamp's theorem). Model checking is PSPACE-complete.

CTL (Clarke-Emerson 1981) reasons about branching computation trees. Every temporal operator is paired with a path quantifier (A = all paths, E = exists path). Model checking is polynomial: $O(|\phi| \cdot |M|)$ via iterative fixpoint computation on the powerset lattice.

CTL* (Emerson-Halpern 1986) subsumes both by allowing free mixing of path quantifiers and temporal operators. CTL and LTL are *incomparable* fragments: CTL can express existential path properties (EG p) that LTL cannot; LTL can express fairness properties (FG p $\rightarrow$ GF q) that CTL cannot.

All three embed into the modal μ-calculus:

Temporal Logic	μ-Calculus Translation
AG ϕ	ν X. $\phi \wedge$ [a]X
EF ϕ	μ X. $\phi \vee \langle$ a $\rangle$ X
AF ϕ	μ X. $\phi \vee$ [a]X
EG ϕ	ν X. $\phi \wedge \langle$ a $\rangle$ X
A(ϕ U ψ)	μ X. $\psi \vee (\phi \wedge$ [a]X $\wedge \langle$ a $\rangle \top)$
E(ϕ U ψ)	μ X. $\psi \vee (\phi \wedge \langle$ a $\rangle$ X)

CTL embeds into the alternation-free fragment (Level 1). CTL* requires alternation depth 2. The Emerson-Lei algorithm computes nested fixpoints in $O(n^d)$ symbolic steps where d is the alternation depth.

3.2 2.2 Signal Temporal Logic (STL)

STL (Maler-Nickovic 2004; Fainekos-Pappas 2009; Donzé-Maler 2010) brings temporal logic to continuous-time, real-valued signals — the natural output of cyber-physical systems.

Syntax: $\phi ::= \mu \mid \neg\, \phi \mid \phi \wedge \psi \mid F_{[a,b]}\, \phi \mid G_{[a,b]}\, \phi \mid \phi\, U_{[a,b]}\, \psi$

where μ is a predicate $f(s(t)) > 0$ over signal values. All temporal operators carry explicit time bounds [a,b].

The Robustness Innovation: The key contribution of STL is *quantitative robustness semantics*. Instead of Boolean satisfaction, the robustness degree $\rho(\phi, s, t) \in \overline{\{}\mathbb{R}\}measureshow\ robustlyasignalsatisfiesorviolatesaspecification:$

- $\rho(\mu, s, t) = f(s(t))$ (raw predicate value)

- $\rho(\neg\ \phi$, s, t) = -$\rho(\phi$, s, t)
- $\rho(\phi \wedge \psi$, s, t) = min($\rho(\phi$, s, t), $\rho(\psi$, s, t))
- $\rho(\phi \vee \psi$, s, t) = max($\rho(\phi$, s, t), $\rho(\psi$, s, t))
- $\rho(G_{[a,b]}\ \phi$, s, t) = $\inf_{t' \in [t+a,\ t+b]}\ \rho(\phi$, s, t')
- $\rho(F_{[a,b]}\ \phi$, s, t) = $\supset_{t' \in [t+a,\ t+b]}\ \rho(\phi$, s, t')

Soundness: $\rho > 0$ implies satisfaction; $\rho < 0$ implies violation. The magnitude $|\rho|$ quantifies the margin.

Lattice-theoretic structure: STL replaces the Boolean lattice {false, true} with the complete lattice $(\overline{\{}\mathbb{R}\}, \leq)$ where min/max replace meet/join, and inf/sup over time intervals replace the fixpoint iteration. This is exactly the pattern propagator networks compute: cells hold lattice values, propagators are monotone functions, and run-to-quiescence computes the fixpoint.

Applications: Online/offline monitoring of CPS, robustness-guided falsification for automotive/avionics testing, control synthesis via optimization-based MPC, and differentiable STL for gradient-based learning.

Key references:

- Maler, O. and Nickovic, D. "Monitoring temporal properties of continuous signals." *FORMATS* 2004.
- Fainekos, G. and Pappas, G. "Robustness of temporal logic specifications for continuous-time signals." *TCS* 410(42), 2009.
- Donzé, A. and Maler, O. "Robust satisfaction of temporal logic over real-valued signals." *FORMATS* 2010.

3.3 2.3 Metric Temporal Logic (MTL)

MTL (Koymans 1990) extends LTL with time-bounded temporal operators interpreted over *timed words* — sequences of events with real-valued timestamps.

$\phi ::= p \mid \neg\ \phi \mid \phi \wedge \psi \mid \phi\ U_I\ \psi$

where I is an interval (e.g., [0,5], (3, ∞)). The decidability landscape is surprisingly delicate:

Domain	Problem	Decidability	Complexity
Finite timed words	Satisfiability	Decidable	Non-primitive recursive
Infinite timed words	Satisfiability	Undecidable	—
Finite timed words	Model checking	Decidable	Non-primitive recursive
Infinite timed words	Model checking	Undecidable	—

The decidability of MTL over finite timed words was established by Ouaknine and Worrell (LICS 2005), resolving a longstanding open question.

MITL (Alur-Feder-Henzinger 1996) bans punctual constraints (exact-time like $U_{=5}$), achieving EXPSPACE-completeness. The slogan: "the cost of punctuality" — exact timing is the source of undecidability.

Connection to timed automata: MTL formulas can be translated to one-clock alternating timed automata (OCATA), which bridge to timed automata (Alur-Dill

1. for model checking.

3.4 2.4 HyperLTL and Hyperproperties

Clarkson and Schneider (JCS 2010) introduced *hyperproperties* — properties of *sets of traces* rather than individual traces. This generalization is necessary because security properties like non-interference and opacity relate multiple execution traces:

HyperLTL (Clarkson-Finkbeiner-Koleini-Rabe-Sanchez, POST 2014) adds trace quantifiers to LTL:

$\psi ::= \forall\ \pi.\ \psi \mid \exists\ \pi.\ \psi \mid \phi$

where ϕ is an LTL formula with indexed atomic propositions a_π.

Expressible properties:

- *Non-interference*: $\forall\ \pi.\ \exists\ \pi'.\ (lo_\pi = lo_{\pi'}) \wedge G(out_\pi \leftrightarrow out_{\pi'})$
- *Observational determinism*: $\forall\ \pi.\ \forall\ \pi'.\ G(l_\pi \leftrightarrow l_{\pi'}) \rightarrow G(o_\pi \leftrightarrow o_{\pi'})$
- *Opacity*: $\forall\ \pi.\ (secret_\pi \rightarrow \exists\ \pi'.\ \neg\ secret_{\pi'} \wedge G(obs_\pi \leftrightarrow obs_{\pi'}))$

Model checking is decidable for the alternation-free fragment but undecidable with quantifier alternation ($\forall\ \exists$). Monitoring alternation-free HyperLTL is feasible; monitoring with alternation is fundamentally limited.

For Prologos: HyperLTL's trace quantifiers mirror the ATMS worldview structure. Universal hyperproperties ($\forall\ \pi$) correspond to "all worldviews satisfy"; existential ($\exists\ \pi$) to "some worldview satisfies." This connection suggests that Prologos could verify hypersecurity properties of session-typed protocols using existing ATMS infrastructure.

3.5 2.5 Probabilistic Temporal Logics

PCTL (Hansson-Jonsson 1994) replaces CTL's path quantifiers A/E with probability thresholds $P_{\geq p}$. Model checking over DTMCs reduces to solving linear equation systems. Over MDPs, it asks whether there exists a policy achieving the probability bound, solvable via linear programming.

CSL (Baier-Haverkort-Hermanns-Katoen 2003) extends PCTL for continuous-time Markov chains with time-bounded until and steady-state operators. Model checking uses uniformisation for transient probability computation.

For propagators: Probabilistic verification replaces the Boolean lattice with [0,1]. Propagator cells holding probability values, refined by probabilistic transition propagators, would compute the same fixpoints as PRISM/Storm. Widening operators for probability lattices (interval abstraction) could accelerate convergence.

3.6 2.6 Spatial and Spatio-Temporal Logics

Caires and Cardelli (2001-2004) developed a *spatial logic for concurrency* with a parallel composition operator and its adjoint (guarantee), mirroring multiplicative linear logic:

- $\phi \mid \psi$: parallel composition (separating conjunction, like * in separation logic)
- $\phi \triangleright \psi$: guarantee (linear implication)

This connects temporal logic directly to *resource reasoning*: spatial reasoning about concurrent processes is fundamentally resource-sensitive. The ambient logic (Cardelli-Gordon 2000) extends this to mobile computation with nested locations.

Topological semantics for modal logic (McKinsey-Tarski 1944) interprets $\Box\phi$ as the interior and $\Diamond\ \phi$ as the closure, identifying S4 modal logic with topological spaces.

3.7 2.7 Quantitative and Weighted Temporal Logics

Discounted LTL (Almagor-Boker-Kupferman, TACAS 2014) replaces Boolean satisfaction with [0,1]-valued satisfaction, discounting temporal distance: F ϕ (w,i) = $\supset_{j \geq i}$ $\lambda^{j-i} \cdot \phi$ (w,j).

Energy/mean-payoff games (Chatterjee-Doyen-Henzinger 2010) combine parity conditions with quantitative resource constraints. Energy parity

games (ensuring resource never drops below zero while satisfying a ω-regular condition) are decidable in NP $\cap$ coNP.

For propagators: Quantitative temporal logics correspond to propagator computations over richer lattices than the Boolean powerset. The energy lattice (Z, $\leq$) with min as meet, the discount lattice ([0,1], $\leq$) with sup/inf — these are all complete lattices amenable to propagator computation. The key insight is that the *same propagator scheduler* handles all lattices; only the cell merge function changes.

4 3. Fixpoint Theory and Propagator Networks

4.1 3.1 The Knaster-Tarski Foundation

The Knaster-Tarski theorem is the mathematical bedrock of both model checking and propagator computation:

Theorem (Knaster 1928, Tarski 1955): Let L be a complete lattice and f : L $\rightarrow$ L a monotone function. Then the set of fixpoints of f forms a complete lattice, with:

- lfp(f) = $\bigcap \backslash$ x $\in$ L | $f(x) \leq$ x $\backslash$ (meet of pre-fixpoints)
- gfp(f) = $\bigcup \backslash$ x $\in$ L | $x \leq$ f(x) $\backslash$ (join of post-fixpoints)

Kleene's theorem strengthens this for ω-continuous functions (those preserving directed suprema): lfp(f) = $\bigsqcup_{\mathrm{i} \geq 0} f^{\mathrm{i}}(\bot), computedbyiteratingfrombottom.Dually, gfp(f) = {}_{\mathrm{i} \geq 0} f^{\mathrm{i}}(\top)$.

Propagator networks are *precisely* Kleene iteration engines:

- Each cell starts at $\bot(ascending)or\top(descending)$
- Each propagator is a monotone function
- `run-to-quiescence` iterates until no cell changes — the fixpoint

4.2 3.2 Approximation Fixpoint Theory (AFT)

Denecker, Marek, and Truszczyński (2000-2004) developed Approximation Fixpoint Theory to unify the semantics of non-monotone reasoning formalisms. The key construction:

Given a lattice L, form the *bilattice* L^2 = L $\times$ L with two orderings:

- *Precision ordering* $\leq_{\mathrm{p}}$: $(\mathrm{x}_1, \mathrm{x}_2) \leq_{\mathrm{p}} (\mathrm{y}_1, \mathrm{y}_2)$ iff $\mathrm{x}_1 \leq \mathrm{y}_1$ and $\mathrm{y}_2 \leq \mathrm{x}_2$ (more precise = tighter approximation)

- *Truth ordering* $\leq_t$: $(x_1, x_2) \leq_t (y_1, y_2)$ iff $x_1 \leq y_1$ and $x_2 \leq y_2$ (more true = higher in both components)

A pair $(x, y) \in L^2$ represents an *approximation*: x is the lower bound (what is certainly true) and y is the upper bound (what is possibly true). An exact element has $x = y$.

Stable operators: An operator $A : L^2 \to L^2$ is an approximation of an operator $O : L \to L$ if for exact (x,x), A(x,x) = (O(x), O(x)). AFT defines:

- *Kripke-Kleene fixpoint*: $\mathrm{lfp}_{\leq_p}(A)$ — the most precise fixpoint in the precision ordering
- *Stable fixpoint*: a fixpoint that is minimal in a certain well-founded iteration
- *Well-founded fixpoint*: always exists and is unique

Unification power: AFT simultaneously captures:

- Stable model semantics (answer set programming)
- Well-founded semantics (logic programming with negation)
- Default logic fixpoints
- Autoepistemic logic expansions

For Prologos: The bilattice L^2 maps directly to a two-cell propagator pattern: one cell for the lower bound (ascending), one for the upper bound (descending). The precision ordering is the product lattice ordering. AFT's approximation operators are exactly cross-cell propagators that enforce the consistency between lower and upper bounds. This gives Prologos a principled way to handle non-monotone operations (like negation-as-failure in the logic engine) within a monotone propagator framework — which is precisely what the stratification controller already does, but without the theoretical framework to generalize it.

4.3 3.3 Belnap's Four-Valued Logic and Bilattices

Belnap (1977) introduced a four-valued logic FOUR = $\{\bot, t, f, \top\} forreasoningwithincompleteandconti$

```
   (both/contradictory)
 / \
t   f
 \ /
   (neither/unknown)
```

This carries two orderings:

- *Knowledge* (information): $\bot < t, f < \top$
- *Truth*: $\mathrm{f} < \bot, \top < t$

Ginsberg (1988) generalized to *bilattices* — structures with two lattice orderings. Fitting (1991-2002) applied bilattices to logic programming, interpreting programs over FOUR to handle incomplete and contradictory information simultaneously.

For Prologos: The session lattice `sess-bot` / concrete / `sess-top` is already a three-valued approximation of FOUR. The ATMS adds a fourth value (contradictory worldview). The connection to bilattice logic programming suggests a path toward handling contradictory constraints gracefully rather than treating contradiction as a hard failure.

4.4 3.4 Mixed Least/Greatest Fixpoints

The central challenge of the μ-calculus is interleaving least and greatest fixpoints. A formula ν X. μ Y. ϕ(X, Y) requires computing:

1. For a given approximation of X, compute the least fixpoint of Y $\mapsto \phi$(X, Y)
2. Use the result to refine X toward its greatest fixpoint
3. Iterate until both X and Y stabilize

The *parity condition* is the general pattern for such interleaving: assign a priority (natural number) to each fixpoint variable, and determine the winner by the parity of the highest priority seen infinitely often. This is why parity games are the decision procedure for the μ-calculus.

Progress measures (Jurdzinski 2000) provide an alternative characterization: a progress measure is a function that assigns to each state a value from a well-founded set, such that the function decreases along certain edges. Computing the least progress measure solves the parity game.

For propagator networks: Mixed lfp/gfp requires cells with *different directions* in the same network:

- Ascending cells (lfp): start at $\bot, refineupviajoin$
- Descending cells (gfp): start at $\top, refinedownviameet$

The scheduler must handle both directions, with nested fixpoints computed inside-out. This is architecturally straightforward — the current BSP/Jacobi scheduler already processes all cells in rounds; adding descending cells requires only changing the merge operation per cell. The deeper question is termination for mixed networks, addressed by Baldan et al.'s fixpoint games (§3.7).

4.5 3.5 Domain Theory and Propagators

Scott domains and dcpos (directed-complete partial orders) provide the denotational semantics that connects to propagator computation:

- A *dcpo* is a partial order where every directed set has a supremum
- A *Scott-continuous* function preserves directed suprema
- The *information ordering* on a dcpo models increasing knowledge

Propagator cells are elements of a dcpo (or complete lattice). The cell's value increases monotonically, representing accumulating information. The merge operation (join) is the directed supremum. Propagators are Scott-continuous functions between dcpos.

This framing connects propagator networks to the rich theory of domain equations, powerdomains, and continuous function spaces. In particular:

- *Powerdomain* constructions (Plotkin, Smyth, Hoare) for nondeterminism correspond to the three ways propagator networks can handle nondeterministic cells (must-may analysis)
- *Bilimit-compact categories* (Amadio-Curien) ensure that recursive domain equations have solutions — relevant for recursive session types interpreted as elements of recursive domains

4.6 3.6 Coinduction and Greatest Fixpoints

Greatest fixpoints are mathematically dual to least fixpoints but operationally different:

- Least fixpoint: finite proof/derivation, inductive definition
- Greatest fixpoint: infinite behavior, coinductive definition

The canonical example is *bisimulation*: two processes are bisimilar iff they are related by the *greatest fixpoint* of the bisimulation functional. This is the coinductive definition: assume the processes are related, and verify that every step preserves the relation.

Coinductive types in type theory: Coinductive types (streams, conatural numbers, infinite trees) represent infinite data. Abel, Pientka, Thibodeau, and Setzer (POPL 2013) introduced *copatterns* for programming with coinductive types: instead of pattern matching on constructors (inductive), copattern matching defines behavior in response to destructors (coinductive).

Productivity checking: For coinductive definitions to be well-defined, they must be *productive* — each observation step must terminate. This is the dual of termination checking for inductive definitions.

For Prologos: Recursive session types with `rec` are naturally coinductive when the protocol loops forever (a server that perpetually accepts requests). The distinction between μ X (inductive, eventually terminating) and ν X (coinductive, potentially infinite) session types maps directly to the lfp/gfp distinction in propagator cells. A coinductive session type should be checked for *productivity* (each protocol step makes progress) rather than *termination* (the protocol eventually ends).

4.7 3.7 Fixpoint Games on Continuous Lattices

Baldan, König, Padoan, and Tsampas (POPL 2019) introduced *fixpoint games on continuous lattices* — a game-theoretic characterization of lattice fixpoints that unifies parity games and abstract interpretation:

Given a system of fixpoint equations $x_i = f_i(x_1, \ldots, x_n)$ over a continuous lattice, they define a two-player game where:

- Player $\exists$ (Existential) chooses witnesses for joins
- Player $\forall$ (Universal) chooses witnesses for meets
- The winner is determined by a parity condition on the fixpoint types (μ/ν)

Key results:

- The game characterizes the solution of the fixpoint equation system
- For finite lattices, the game reduces to classical parity games

- For infinite lattices, the game provides a sound approximation framework

This directly addresses the question of how propagator networks compute nested fixpoints: the scheduler can be viewed as *playing the fixpoint game*, with ascending cell updates as Existential moves and descending cell updates as Universal moves.

4.8 3.8 Widening and Narrowing

Cousot and Cousot (1977, 1992) introduced widening (∇) and narrowing (Δ) operators to ensure termination of fixpoint iteration over infinite-height lattices:

- *Widening* $\nabla : L \times L \to L$ accelerates ascending chains by jumping ahead, guaranteeing termination at the cost of precision (the result is an over-approximation of the lfp)
- *Narrowing* $\Delta : L \times L \to L$ improves precision by descending from the widened result, without losing the termination guarantee

Standard variants include:

- *Delayed widening*: apply standard iteration for k steps before widening
- *Threshold widening*: widen to predefined thresholds (e.g., 0, 1, ∞)
- *Guided widening*: use the property being checked to guide the widening

For Prologos: Widening is already implemented via the `Widenable` trait and `net-add-widening-propagator`. The connection to abstract interpretation suggests widening strategies informed by the temporal property being verified: if checking AF($x < 10$), widen the interval lattice with threshold 10.

5 4. Runtime Verification and Monitoring

5.1 4.1 Three-Valued Semantics (LTL3)

Runtime verification faces a fundamental challenge: LTL is defined over infinite traces, but monitors observe only finite (growing) prefixes. Bauer, Leucker, and Schallhart (TOSEM 2011) introduced LTL3 with three values:

- **T** (true): every possible infinite continuation satisfies the property
- **F** (false): every possible infinite continuation violates the property
- **?** (inconclusive): some continuations satisfy, some violate

Formally: $[u \models \phi]_3 = T$ iff $\forall\, w \in \Sigma\hat{}\omega : u \cdot w \models \phi$.

The monitor construction produces a *minimal deterministic finite-state machine* that reads finite traces and outputs B3 verdicts. Extended to RV-LTL with four values: {T, F, T_p (presumably true), F_p (presumably false)} based on behavioral pattern extrapolation.

Monitorable properties: A property ϕ is monitorable if for every finite trace u, there exists a finite extension v such that $u \cdot v$ yields a definitive verdict. This class is *strictly larger* than safety $\cup$ co-safety. Safety properties are finitely refutable; co-safety are finitely verifiable; liveness properties are generally not finitely monitorable.

Monitorable fragments of the μ-calculus: The alternation-free fragment (no mutual recursion between μ and ν) yields monitorable properties. Research on Hennessy-Milner logic with recursion (a μ-calculus variant) has established an expressiveness hierarchy of monitorable fragments.

5.2 4.2 Monitoring Tools and Architectural Patterns

The runtime verification community has developed a rich ecosystem of tools:

Stream-based monitors:

- *Lola* (D'Angelo et al., TIME 2005): Declarative stream equations defining output streams as functions of inputs. Bounded memory for syntactically well-behaved specifications.
- *RTLola* (Finkbeiner et al., CAV 2020): Real-time extension with asynchronous timestamped streams and temporal window aggregation. Compiles to FPGA for hardware monitoring. Deployed on DLR ARTIS autonomous rotorcraft.
- *TeSSLa*: Non-synchronized real-time streams with recursive aggregation.

Embedded/hardware monitors:

- *Copilot* (Pike et al., RV 2010): Haskell EDSL generating hard real-time C99 monitors running in constant memory and constant time. Developed under NASA contract for avionics.

- *R2U2* (Reinbacher et al., FMSD 2017): FPGA-based monitoring with LTL/MLTL observers and integrated Bayesian health diagnosis. Zero software overhead.

Parametric monitors:

- *JavaMOP* (Jin et al., ICSE 2012): Monitoring-Oriented Programming with AspectJ code generation. Supports parametric properties relating specific object instances.
- *MarQ* (Reger et al., TACAS 2015): Quantified Event Automata for parametric monitoring with efficient trace slicing.

Pattern	Description	Examples
Inline	Monitor woven into system code	JavaMOP (AspectJ)
Sidecar	Monitor observes traces externally	Copilot, R2U2
Stream pipeline	Declarative stream equations	Lola, RTLola, TeSSLa
Hardware parallel	FPGA-based constant-time monitoring	R2U2, RTLola-FPGA

For Prologos: The stream-based monitoring pattern maps directly to propagator networks. Lola's stream equations *are* propagator wiring: each output stream is a cell whose value is determined by propagators reading input streams. The "compile temporal spec to monitor" pipeline is the same as "compile temporal spec to propagator network." The difference is that Prologos monitors would run at *type-checking time* (static analysis) or at *runtime* (dynamic monitoring), using the same propagator infrastructure for both.

5.3 4.3 Distributed Runtime Verification

Monitoring distributed systems introduces challenges from partial ordering and concurrent observation:

Lamport timestamps and vector clocks: Establish partial ordering of events across processes. Vector clocks (Fidge 1988, Mattern 1989) precisely characterize the happens-before relation.

Chase-Garg lattice of consistent cuts (1995): The set of consistent global states forms a finite distributive lattice under inclusion. Predicate detection on this lattice ranges from $O(n^2)$ for linear predicates (meet-closed) to NP-complete for general predicates.

Decentralized monitoring (Bauer-Falcone, FMSD 2016): Decomposing LTL specifications into sub-formulae distributed among local monitors with partial observation.

For Prologos: Distributed session types (multi-party protocols) require distributed verification. The lattice of consistent cuts is itself a complete lattice, and predicate detection on it is a fixpoint computation — naturally amenable to propagator-based evaluation. Prologos's multi-party session types could embed distributed monitors that check protocol compliance across endpoints.

5.4 4.4 Predictive Runtime Verification

Predictive RV detects *potential* violations that haven't occurred yet by analyzing causal models:

- *Happens-before analysis*: Explore alternative interleavings consistent with the observed partial order
- *Maximal causal models*: Consider all causally compatible executions
- *SMT-based approaches*: Encode the space of possible interleavings as SMT constraints and check for violations

This connects to the ATMS: each consistent worldview is an alternative interleaving, and checking a property across all worldviews is predictive verification.

5.5 4.5 Shield Synthesis

Bloem, Könighofer, and Ehlers introduced *shields* — reactive systems that enforce safety by observing the system's output and correcting unsafe actions:

- *Pre-shields*: Intercept controller inputs, restricting available actions to those guaranteed safe
- *Post-shields*: Intercept controller outputs, correcting unsafe actions to safe alternatives
- Formalized as safety games between the shield and the environment

Safe reinforcement learning through shielding: The shield constrains the RL agent's action space, preventing unsafe exploration while maintaining learning effectiveness. The shield is synthesized from a temporal specification and a model of the environment.

For Prologos: Shield synthesis from session types would produce runtime monitors that *enforce* protocol compliance, not just detect violations. A session-typed channel with a shield would automatically correct protocol deviations — a form of self-healing communication. This extends the static verification story with dynamic enforcement.

6 5. Session Types, Linear Logic, and Verification

6.1 5.1 The Curry-Howard Correspondence for Sessions

The Caires-Pfenning correspondence (CONCUR 2010) and Wadler's "Propositions as Sessions" (ICFP 2012) establish a deep isomorphism between classical linear logic and session types:

Linear Logic	Session Type
A $\otimes$ B	Send A then continue as B
A B	Receive A then continue as B
A $\oplus$ B	Internal choice (select A or B)
A \	B
External choice (offer A or B)	
!A	Server (replicated service)
?A	Client (replicated usage)
Cut elimination	Communication (β-reduction)

Key theorem: Deadlock freedom follows from cut elimination in linear logic. Well-typed processes do not deadlock because their typing derivations correspond to valid linear logic proofs, and cut elimination preserves validity.

6.2 5.2 Recursive Session Types: μ vs ν

Recursive session types come in two flavors, mirroring the lfp/gfp distinction:

- **Inductive** (μ): The protocol eventually terminates. The recursive type μ X. !Msg. ?Ack. X describes a bounded interaction that must reach a base case. Checked by termination/productivity analysis.
- **Coinductive** (ν): The protocol may run forever. The recursive type ν X. !Req. ?Resp. X describes a persistent server that perpetually accepts requests. Checked by productivity: each protocol step is responsive.

The distinction is operationally critical: inductive session types correspond to clients (finite interaction); coinductive session types correspond to servers (persistent service). Prologos's `rec` keyword currently treats all recursion as potentially infinite; distinguishing μ-rec from ν-rec would enable more precise verification.

6.3 5.3 Multiparty Session Types

Honda, Yoshida, and Carbone (POPL 2008) extended binary session types to multi-party protocols via *global types* — protocol specifications describing the interaction among all participants:

1. *Global type*: Describes the protocol from a bird's-eye view
2. *Projection*: Each participant's local type is projected from the global type
3. *Deadlock freedom*: If all participants follow their projected local types, the system is deadlock-free

Recent extensions include:

- *Timed multiparty session types* (ECOOP 2024): Adding timing constraints
- *Crash-stop multiparty session types* (ECOOP 2023): Handling participant failures
- *Behavioral contracts* (Castagna-Gesbert-Padovani 2009): Subtyping for session types ensuring safe substitution
- *Gradual session types* (Igarashi et al. 2017): Blending static and dynamic session type checking

6.4 5.4 Session Types and Model Checking

Lange, Tuosto, and Yoshida developed Communicating Session Automata (CSA) — a direct bridge between session types and model checking:

- Global types are translated to communicating finite-state machines (CFSMs)
- Properties (deadlock freedom, protocol conformance) are checked via model checking on the product CFSM

- The model checking can verify temporal properties beyond what type checking alone can ensure

This is precisely the bridge Prologos aims to build, but using propagator networks instead of CFSMs as the verification substrate.

6.5 5.5 Actris: Session Types Meet Separation Logic

Hinrichsen, Bengtson, and Krebbers (POPL 2020) introduced Actris — *Dependent Separation Protocols* within the Iris framework. This merges session types with higher-order separation logic, where protocol steps can depend on separation logic propositions about heap state.

Actris 2.0 (LMCS 2022) adds subprotocols (session-type subtyping) for compositional reasoning. This is the most sophisticated integration of session types with formal verification to date, and demonstrates that session types and separation logic are not competing but complementary verification approaches.

7 6. Formal Verification Frontiers

7.1 6.1 Separation Logic and Iris

Iris (Jung et al., POPL 2015) is a higher-order concurrent separation logic framework mechanized in Coq, built on three orthogonal pillars:

1. **Resource Algebras (Cameras)**: Generalized partial commutative monoids with a validity predicate — the algebraic foundation for resource reasoning

2. **Invariants**: Named propositions that hold at all times, with impredicative invariant opening

3. **Ghost State**: Logical state not in the physical program, governed by resource algebras and frame-preserving updates

The *later modality* $(\triangleright P) resolves circularity in impredicative definitions via guarded recursion.$

RustBelt (Jung et al., POPL 2018) proves Rust's type system sound, including unsafe code, using Iris's lifetime logic. **RefinedRust** (Sammler et al., PLDI

1. layers refinement types on Rust within Iris, with machine-checked proofs.

7.2 6.2 Refinement Types

Liquid types (Rondon-Kawaguchi-Jhala, PLDI 2008) combine HM type inference with predicate abstraction: types $\{v : T \mid p\}$ where p is drawn from a decidable qualifier language. Subtyping reduces to SMT implication checking.

Refinement reflection (Vazou et al., POPL 2018) enables complete verification by reflecting function definitions into their output refinement types.

F* (Swamy et al.) combines refinement types with an *indexed effect system* organized as a lattice of effects, enabling modular verification of stateful programs.

For Prologos: Refinement types could extend Prologos's dependent types with SMT-backed lightweight verification. The qualifier lattice in liquid type inference is a complete lattice amenable to propagator computation — qualifier inference *is* fixpoint computation over the qualifier lattice.

7.3 6.3 Algebraic Effects and Handlers

Algebraic effects (Plotkin-Power 2001-2003) model computational effects as operations of an algebraic theory. Effect handlers (Plotkin-Pretnar, ESOP 2009) generalize exception handlers with access to delimited continuations.

Key languages: Eff, Frank (where handlers are the universal abstraction), Koka (row-polymorphic effect types compiling to C). Recent work (Sekiyama-Tsukada, POPL 2024) combines refinement types with algebraic effects.

Connection to session types: Communication operations (send, receive) can be modeled as algebraic effects; session types then become effect types constraining operation ordering. Both algebraic effects (via free monads) and session types (via indexed monads) unify under graded monadic frameworks.

7.4 6.4 Reactive Synthesis

Given a temporal specification, *synthesize* an implementation automatically:

- **Church's problem** (1957): Realized by the Büchi-Landweber theorem (1969) for ω-regular specifications
- **GR(1) synthesis** (Bloem-Jobstmann-Piterman-Pnueli-Sa'ar, JCSS 2012): Efficient $O(n \cdot m \cdot N^2)$ synthesis for the practically important generalized reactivity(1) fragment

- **LTL synthesis**: 2EXPTIME-complete (Pnueli-Rosner 1989); Safraless approaches (Kupferman-Vardi, FOCS 2005) avoid determinization
- **Bounded synthesis** (Finkbeiner-Schewe, ATVA 2007): Restrict to implementations with at most k states; reduces to SAT

For Prologos: Reactive synthesis from session type specifications is a natural extension of the propagator-based verification story. Given temporal properties on session types, synthesize process implementations that satisfy them. The propagator network supports this through its *bidirectional* nature — constraints flow in both directions, enabling constructive derivation of implementations from specifications.

7.5 6.5 Coalgebraic Semantics

Coalgebras provide a unified categorical treatment of transition systems and temporal logics:

- A coalgebra for functor F is a morphism $c : X \to F(X)$
- Different F capture different system types: P(X) for nondeterminism, D(X) for probability, $(2 \times X)^A$ for deterministic automata
- Coalgebraic modal logic uses predicate liftings to define modal operators parametrically in the functor

This enables *parametric verification*: fixing the functor determines the system type, and results transfer across functor choices. Stone duality connects coalgebraic logic to topology, yielding representation theorems and completeness results.

For Prologos: Prologos's propagator network is itself a coalgebra — the network state transforms via propagator application. Temporal properties of the network (convergence, stability, monotonicity) can be expressed in coalgebraic modal logic. The terminal coalgebra (greatest fixpoint) of the network functor represents the "fully resolved" type/session/effect assignment.

7.6 6.6 Capability-Based Security Verification

Object-capability (oCap) security restricts authority propagation to explicit capability passing. Formal verification includes:

- **Cerise** (Georges et al., JACM 2023): Verifying capability machine programs using Iris, proving security even with untrusted code
- **CHERI** (Watson et al.): Hardware-enforced capabilities with unforgeable tagged pointers; VeriCHERI provides RTL-level verification
- **WebAssembly**: Capability-based module system with explicit import/export surfaces

Connection to linear types: Capability security and linear types share deep structure — both enforce resource discipline. Affine capabilities (usable at most once) are linear resources. CHERI's hardware tag enforces a form of linearity. Cerise explicitly uses Iris's resource algebras (substructural reasoning) to verify capability programs.

For Prologos: Prologos's linear types (QTT) already provide resource control; adding a capability-based module system would formalize authority propagation. The propagator network can verify capability confinement properties: a capability that should not escape a scope is a linear resource that must not be sent outside its boundary — checkable via the session type checker.

8 7. The Propagator Network as Universal Verification Substrate

8.1 7.1 The Fundamental Observation

The preceding survey reveals a common mathematical structure across all formalisms:

Every verification formalism discussed in this document computes fixpoints over complete lattices.

Formalism	Lattice	Fixpoint Type
CTL model checking	$(2^S, \subseteq)$	lfp and gfp
μ-calculus	$(2^S, \subseteq)$	nested lfp/gfp
STL robustness	$(\bar{\{}\mathbb{R}\}, \leq)$	inf/sup over time
Abstract interpretation	Abstract domain via Galois conn.	lfp with widening
AFT / bilattice	L^2 with precision ordering	Kripke-Kleene/stable
PCTL probabilistic	$([0,1], \leq)$	lfp via lin. eqs.
Energy games	$(Z, \leq)$	lfp/gfp combined
Type inference	Type lattice	lfp via unification
Session type checking	Session lattice	lfp (currently)
LTL3 monitoring	$(\{T, F, ?\}, \leq_k)$	lfp (trace prefixes)
Liquid type inference	Qualifier lattice	lfp via abstraction
Reactive synthesis	Game lattice	nested lfp/gfp

Propagator networks are a *general-purpose engine* for lattice fixpoint computation. The network infrastructure — cells, propagators, scheduler, ATMS — is agnostic to the specific lattice. Adding a new verification capability means:

1. Define the lattice (implement the `Lattice` trait)
2. Define the propagators (monotone functions between cells)
3. Wire them into the existing network
4. `run-to-quiescence` computes the fixpoint

8.2 7.2 What Prologos Already Has (Infrastructure Inventory)

From `propagator.rkt` and the existing verification layers:

Component	What It Provides	Verification Role
Cells (ascending)	Lattice-valued mutable state	State labeling, property cells
Monotone propagators	Functions between cells	Transition relation, operators
BSP/Jacobi scheduler	Parallel fixpoint iteration	Model checking algorithm
ATMS	Multi-hypothesis reasoning	Path quantification (A/E)
Nogoods	Inconsistent assumption sets	Impossible paths / counterexamples
Galois connections	Cross-domain abstraction	Abstract model checking
Widening (`Widenable`)	Accelerated convergence	Termination for infinite lattices
Threshold propagators	Wait-for-value patterns	Stratified evaluation barriers
Cross-domain propagators	Inter-lattice bridges	Multi-domain verification

8.3 7.3 What Would Complete the Picture

8.3.1 7.3.1 Greatest Fixpoint Support (Descending Cells)

Currently all cells start at $\bot andrefineupward(lfp).Safetypropertiesandcoinductivedefinitionsrequir$ $cellsstartingat\top andrefiningdownward(gfp)$.

Implementation sketch:

1. Add `cell-direction` flag: `:ascending` (default) or `:descending`
2. Descending cells use `meet` instead of `join` for `net-cell-write`
3. Contradiction for descending cells is reaching $\bot(dualofascending\top)$
3. The scheduler processes both directions; nested fixpoints stabilize inside-out

This is architecturally straightforward. The session lattice already has both join and meet. The extension adds the dual direction to the scheduler.

8.3.2 7.3.2 Temporal Property Specification Language

A user-facing language for specifying temporal properties that compiles to propagator wiring (developed in detail in `PROPAGATORS_AS_MODEL_CHECKERS.org` §4):

```
property pin-safe
  :session Pin-Guard
  :always [not [and :pinned :gc-collecting]]

property request-response
  :session CapTP-Steady
  :always [implies [sent :call] [eventually [received :return]]]
```

Each keyword maps to a fixpoint construction:

- `:always` → ν X. $\phi \wedge$ [a]X (greatest fixpoint)
- `:eventually` → μ X. $\phi \vee$ [a]X (least fixpoint)
- `:until` → μ X. $\psi \vee (\phi \wedge$ [a]X) (least fixpoint)

8.3.3 7.3.3 Approximation Fixpoint Theory Integration

Using AFT's bilattice construction L^2 for handling non-monotone operations:

- Lower bound cell (what is certainly true) + upper bound cell (what is possibly true)
- Stable operator applied to the pair
- Generalizes the stratification controller to arbitrary non-monotone operators

8.3.4 7.3.4 Runtime Monitoring Compilation

Temporal properties that cannot be verified statically (due to undecidability or state-space explosion) compile to *runtime monitors* — propagator networks that run alongside the program:

- Stream-based monitoring (Lola-style) maps directly to propagator wiring
- The same temporal specification language works for both static and dynamic verification
- Shields (enforcement monitors) correct unsafe behavior rather than just detecting it

8.3.5 7.3.5 Quantitative Verification Support

Replace the Boolean lattice with richer domains:

- $(\overline{\{}\mathbb{R}\}, \leq)$ for STL robustness monitoring
- ([0,1], $\leq$) for probabilistic verification
- (Z, $\leq$) for energy/resource constraints

The propagator scheduler is lattice-agnostic; only the cell merge function changes.

8.4 7.4 The Verification Layer Cake (Extended)

```
Layer 7: Hyperproperties          (HyperLTL: information flow, non-interference)
     |                              → ATMS worldview quantification
Layer 6: Temporal Properties      (CTL/LTL/-calculus: safety, liveness, fairness)
     |                              → Fixpoint propagators (ascending + descending)
Layer 5: Runtime Monitors         (Stream-based monitoring, shields)
     |                              → Compiled propagator sub-networks
Layer 4: Effect Ordering          (Causal model: deadlock detection)
     |                              → Galois connection to session domain
Layer 3: Session Types            (Protocol state machines: duality, completeness)
     |                              → Session lattice, decomposition propagators
Layer 2: Linear Types (QTT)       (Resource tracking: use-once, capabilities)
     |                              → Multiplicity lattice, QTT propagators
Layer 1: Dependent Types          (Value-dependent properties: indexed types)
     |                              → Type lattice, unification propagators
Layer 0: Propagator Network       (Monotone fixpoint computation: universal substrate)
         + ATMS (path quantification)
         + Galois Connections (abstraction)
         + Widening (convergence acceleration)
         + AFT/Bilattice (non-monotone recovery)
```

Every layer builds on Layer 0. Every new layer adds cells and propagators to the *same network*. The cross-layer interactions happen through the shared substrate. A temporal property that references type information, session state, effect ordering, and resource tracking is verified in a single `run-to-quiescence` pass because all layers share the network.

9 8. Cross-Cutting Themes

9.1 8.1 The Lattice Universality Principle

The recurring theme across all formalisms is that *verification is lattice computation*. Different formalisms choose different lattices and different fixpoint operators, but the computational pattern is invariant:

1. Define a complete lattice
2. Define monotone functions over it
3. Compute fixpoints (least, greatest, or nested)

4. Interpret the fixpoint as a verification result

Propagator networks are the most general realization of this pattern: they are *parameterized* by the lattice and the functions, and the scheduler computes fixpoints for *any* instantiation.

9.2 8.2 The Duality of Safety and Liveness

Safety (*nothing bad happens*) and liveness (*something good eventually happens*) are the two fundamental verification concerns. They correspond exactly to the two fixpoint types:

Concern	Fixpoint	Cell Direction	Iteration
Safety	gfp (ν)	Descending	Start at $\top, refinedown$
Liveness	lfp (μ)	Ascending	Start at $\bot, refineup$

The Alpern-Schneider decomposition theorem (1985) states that every temporal property is the intersection of a safety and a liveness property. In propagator terms: every temporal property can be decomposed into an ascending cell (liveness component) and a descending cell (safety component), with propagators connecting them.

9.3 8.3 Static vs. Dynamic Verification

The same propagator infrastructure supports both static (compile-time) and dynamic (runtime) verification:

Aspect	Static Verification	Dynamic Verification
Input	Type/session annotations	Runtime events/signals
Lattice	Type/session lattice	Trace prefix lattice / [0,1]
Completeness	May be incomplete	Always reflects actual behavior
Overhead	Compile-time only	Runtime performance cost
Expressiveness	Decidable properties only	Any monitorable property
Implementation	Propagators in type checker	Propagators in runtime system

The key insight is that *the propagator wiring is the same*. A temporal property specification compiles to the same propagator structure whether it runs during type checking or during execution. The difference is only in the source of cell updates: type annotations vs. runtime events.

9.4 8.4 The Abstraction-Refinement Paradigm

Abstract interpretation, CEGAR, and Galois connections are all manifestations of the same pattern: compute on a simpler (abstract) domain, and refine when the abstraction is too coarse.

In propagator terms:

1. Create abstract cells connected to concrete cells via Galois connection propagators
2. Compute the fixpoint on the abstract domain (faster, may be imprecise)
3. If the abstract result is inconclusive, refine the abstraction (add finer abstract cells) and re-run
4. The propagator network handles this incrementally — only affected cells are re-evaluated

9.5 8.5 Resource Discipline as Universal Theme

Across the formalisms, *resource discipline* — controlling who can use what, when, and how many times — is the universal verification theme:

- **Linear types (QTT)**: Resources used exactly once (or controlled multiplicity)
- **Session types**: Communication channels used according to protocol
- **Capabilities**: Authorities that cannot be forged
- **Energy constraints**: Resources consumed/produced along execution
- **Separation logic**: Heap resources owned exclusively or shared under invariant
- **Algebraic effects**: Computational effects requiring explicit handling

All enforce the same principle at different granularities. Prologos's combination of QTT + session types + capability security already covers the first three; extending to energy constraints and separation-logic reasoning would complete the resource verification picture.

10 9. Research Program

10.1 9.1 Immediate (Infrastructure Extensions)

1. **Greatest fixpoint cells**: Add `:descending` cell direction to `propagator.rkt`. Validate with bisimulation checking as the first application.

2. **Mixed ascending/descending networks**: Verify that the BSP scheduler correctly handles mixed-direction cells with nested fixpoint computation. Test with Level 2 alternation (ν X. μ Y. ϕ).

3. **STL robustness cells**: Implement $\overline{\{}\mathbb{R}\}-valuedcellswithinf/supmerge.Validatewithsimplecon$

10.2 9.2 Medium-Term (New Verification Capabilities)

1. **Temporal property language**: Design and implement the `property` keyword with temporal operators (:always, :eventually, :until, :within). Compile to propagator wiring.

2. **AFT integration**: Implement bilattice L^2 cell pairs for non-monotone operator recovery. Generalize the stratification controller.

3. **Runtime monitor compilation**: Compile temporal specifications to Lola-style stream monitors that can run alongside Prologos programs.

4. **Multiparty session type verification**: Extend session type propagators to handle global types with projection and multi-participant deadlock checking.

10.3 9.3 Long-Term (Research Frontiers)

1. **Reactive synthesis from session types**: Given temporal properties on a session type, synthesize conforming process implementations. Leverage the bidirectional nature of propagator networks.

2. **Probabilistic session types**: Session types with probabilistic branching and PCTL-style probability bounds on protocol properties.

3. **Hyperproperty verification**: Leverage ATMS worldview quantification to verify HyperLTL properties of session-typed protocols (information flow, non-interference).

4. **Coinductive session types**: Distinguish μ-rec (inductive/terminating) from ν-rec (coinductive/persistent) and check productivity rather than termination for persistent servers.

5. **Capability confinement verification**: Verify that capabilities do not escape their intended scope, using propagator-based reachability analysis on the session type's communication graph.

6. **Shield synthesis for session types**: Synthesize runtime enforcement shields from session type specifications, producing self-healing communication channels.

11 10. References

11.1 Modal μ-Calculus and Fixpoint Theory

- Kozen, D. "Results on the propositional μ-calculus." *TCS* 27(3), 1983.
- Walukiewicz, I. "Completeness of Kozen's axiomatisation." *Information and Computation* 157(1-2), 2000.
- Janin, D. and Walukiewicz, I. "On the expressive completeness of the propositional μ-calculus with respect to MSO." *CONCUR* 1996.
- Bradfield, J. "The modal μ-calculus alternation hierarchy is strict." *CONCUR* 1996.
- Calude, C., Jain, S., Khoussainov, B., Li, W., and Stephan, F. "Deciding parity games in quasipolynomial time." *STOC* 2017.
- Jurdzinski, M. "Small progress measures for solving parity games." *STACS* 2000.
- Jurdzinski, M. and Lazic, R. "Succinct progress measures for solving parity games." *LICS* 2017.
- Baldan, P., König, B., Padoan, T., and Tsampas, C. "Fixpoint games on continuous lattices." *POPL* 2019.
- Emerson, E.A. and Jutla, C. "Tree automata, mu-calculus and determinacy." *FOCS* 1991.

11.2 Temporal Logics

- Pnueli, A. "The temporal logic of programs." *FOCS* 1977.
- Clarke, E., Emerson, E., and Sistla, A. "Automatic verification of finite-state concurrent systems using temporal logic specifications." *ACM TOPLAS* 8(2), 1986.
- Emerson, E. and Halpern, J. "'Sometimes' and 'not never' revisited: on branching versus linear time temporal logic." *JACM* 33(1), 1986.
- Maler, O. and Nickovic, D. "Monitoring temporal properties of continuous signals." *FORMATS* 2004.
- Fainekos, G. and Pappas, G. "Robustness of temporal logic specifications for continuous-time signals." *TCS* 410(42), 2009.
- Donzé, A. and Maler, O. "Robust satisfaction of temporal logic over real-valued signals." *FORMATS* 2010.
- Koymans, R. "Specifying real-time properties with metric temporal logic." *RTS* 2(4), 1990.
- Alur, R., Feder, T., and Henzinger, T. "The benefits of relaxing punctuality." *JACM* 43(1), 1996.
- Ouaknine, J. and Worrell, J. "On the decidability of metric temporal logic." *LICS* 2005.

11.3 Hyperproperties and Information Flow

- Clarkson, M. and Schneider, F. "Hyperproperties." *JCS* 18(6), 2010.
- Clarkson, M., Finkbeiner, B., Koleini, M., Rabe, M., and Sanchez, C. "Temporal logics for hyperproperties." *POST* 2014.

11.4 Probabilistic and Quantitative Verification

- Hansson, H. and Jonsson, B. "A logic for reasoning about time and reliability." *Formal Aspects of Computing* 6(5), 1994.
- Baier, C., Haverkort, B., Hermanns, H., and Katoen, J.-P. "Model-checking algorithms for continuous-time Markov chains." *IEEE TSE* 29(6), 2003.

- Kwiatkowska, M., Norman, G., and Parker, D. "PRISM 4.0: Verification of probabilistic real-time systems." *CAV* 2011.
- Almagor, S., Boker, U., and Kupferman, O. "Discounting in LTL." *TACAS* 2014.
- Chatterjee, K. and Doyen, L. "Energy parity games." *ICALP* 2010.
- Kaminski, B., Katoen, J.-P., Matheja, C., and Olmedo, F. "Weakest precondition reasoning for expected runtimes of randomized algorithms." *JACM* 2018.

11.5 Runtime Verification

- Bauer, A., Leucker, M., and Schallhart, C. "Runtime verification for LTL and TLTL." *ACM TOSEM* 20(4), 2011.
- Leucker, M. and Schallhart, C. "A brief account of runtime verification." *JLAP* 78(5), 2009.
- D'Angelo, B. et al. "LOLA: Runtime monitoring of synchronous systems." *TIME* 2005.
- Finkbeiner, B. et al. "RTLola cleared for take-off: Monitoring autonomous aircraft." *CAV* 2020.
- Pike, L. et al. "Copilot: A hard real-time runtime monitor." *RV* 2010.
- Reinbacher, T. et al. "R2U2: Monitoring and diagnosis of security threats for unmanned aerial systems." *FMSD* 2017.
- Jin, D., Meredith, P., Lee, C., and Rosu, G. "JavaMOP: Efficient parametric runtime monitoring framework." *ICSE* 2012.

11.6 Approximation Fixpoint Theory and Bilattices

- Denecker, M., Marek, V., and Truszczyński, M. "Approximations, stable operators, well-founded fixpoints and applications in nonmonotonic reasoning." *Logic-Based AI* 2000.
- Denecker, M., Marek, V., and Truszczyński, M. "Ultimate approximation and its application in nonmonotonic knowledge representation systems." *Information and Computation* 192(1), 2004.

- Belnap, N. "A useful four-valued logic." *Modern Uses of Multiple-Valued Logic* 1977.
- Ginsberg, M. "Multivalued logics: A uniform approach to reasoning in AI." *Computational Intelligence* 4, 1988.
- Fitting, M. "Bilattices and the semantics of logic programming." *JLP* 11(2), 1991.

11.7 Abstract Interpretation

- Cousot, P. and Cousot, R. "Abstract interpretation: A unified lattice model for static analysis." *POPL* 1977.
- Cousot, P. and Cousot, R. "Comparing the Galois connection and widening/narrowing approaches to abstract interpretation." *PLILP* 1992.
- Clarke, E., Grumberg, O., and Long, D. "Model checking and abstraction." *ACM TOPLAS* 16(5), 1994.

11.8 Session Types and Linear Logic

- Caires, L. and Pfenning, F. "Session types as intuitionistic linear propositions." *CONCUR* 2010.
- Wadler, P. "Propositions as sessions." *JFP* 24(2-3), 2014.
- Honda, K., Yoshida, N., and Carbone, M. "Multiparty asynchronous session types." *POPL* 2008.
- Hinrichsen, J., Bengtson, J., and Krebbers, R. "Actris: Session-type based reasoning in separation logic." *POPL* 2020.

11.9 Separation Logic

- Jung, R. et al. "Iris: Monoids and invariants as an orthogonal basis for concurrent reasoning." *POPL* 2015.
- Jung, R. et al. "Iris from the ground up." *JFP* 28, 2018.
- Jung, R. et al. "RustBelt: Securing the foundations of the Rust programming language." *POPL* 2018.
- Sammler, M. et al. "RefinedRust: A type system for high-assurance verification of Rust programs." *PLDI* 2024.

11.10 Refinement Types and Verification

- Rondon, P., Kawaguchi, M., and Jhala, R. "Liquid types." *PLDI* 2008.
- Vazou, N. et al. "Refinement reflection: Complete verification with SMT." *POPL* 2018.
- Swamy, N. et al. "Dependent types and multi-monadic effects in F*." *POPL* 2016.

11.11 Algebraic Effects

- Plotkin, G. and Pretnar, M. "Handlers of algebraic effects." *ESOP* 2009.
- Lindley, S., McBride, C., and McLaughlin, C. "Do be do be do." *POPL* 2017.

11.12 Reactive Synthesis

- Bloem, R., Jobstmann, B., Piterman, N., Pnueli, A., and Sa'ar, Y. "Synthesis of reactive(1) designs." *JCSS* 78(3), 2012.
- Pnueli, A. and Rosner, R. "On the synthesis of a reactive module." *POPL* 1989.
- Finkbeiner, B. and Schewe, S. "Bounded synthesis." *ATVA* 2007.

11.13 Capability Security

- Swasey, D., Garg, D., and Dreyer, D. "Robust and compositional verification of object capability patterns." *OOPSLA* 2017.
- Georges, A. et al. "Cerise: Program verification on a capability machine in the presence of untrusted code." *JACM* 2023.

11.14 Coalgebraic Methods

- Kupke, C. and Pattinson, D. "Coalgebraic semantics of modal logics: An overview." *TCS* 2011.
- Hasuo, I. "Tracing anonymity with coalgebras." PhD Thesis, Radboud University, 2008.

11.15 Domain Theory and Coinduction

- Abel, A., Pientka, B., Thibodeau, D., and Setzer, A. "Copatterns: Programming infinite structures by observations." *POPL* 2013.
- Amadio, R. and Curien, P.-L. *Domains and Lambda-Calculi*. Cambridge, 1998.

11.16 Propagator Networks

- Radul, A. "Propagation Networks: A Flexible and Expressive Substrate for Computation." PhD Thesis, MIT, 2009.
- Radul, A. and Sussman, G. "The Art of the Propagator." MIT CSAIL, 2009.
- De Kleer, J. "An assumption-based TMS." *Artificial Intelligence* 28(2), 1986.

11.17 Prologos Project Internal Documents

- "Propagator Networks as Model Checkers." `docs/research/2026-03-07_PROPAGATORS_AS_MODEL` 2026.
- "The Categorical Structure of Layered Recovery." `docs/research/2026-03-13_LAYERED_RECOVER` 2026.
- "Propagator Networks: A Comprehensive Survey." `docs/research/PROPAGATOR_NETWORKS.org`. 2026.